MEMORIA DE ACTIVIDAD

# Actividad 2.3 — Búsqueda local

*Implementación y optimización de Hill Climbing y Simulated Annealing para la minimización de una ruta de reparto (TSP)*

**Asignatura:** Fundamentos de Inteligencia Artificial

**Estudiante:** Julissa Marin Flor

**Docente:** Javier García Lara

**Institución:** Universidad Politécnica de Atlautla

**Fecha:** 22 de octubre de 2026

## 1. Enunciado

Implementar los algoritmos de búsqueda local Hill Climbing y Simulated Annealing para optimizar una función real, reportando: enunciado, función, objetivo, vecindad, parámetros (T, iteraciones, n), gráfica de convergencia y comparación de resultados.

## 2. Problema seleccionado

Se seleccionó el problema del viajante de comercio (Traveling Salesperson Problem, TSP): encontrar el recorrido que pasa por todas las paradas de entrega saliendo y volviendo al almacén, minimizando la distancia total recorrida. Es un problema clásico de optimización combinatoria con un espacio de búsqueda muy grande, ideal para evaluar algoritmos de búsqueda local.

## 3. Función, objetivo y representación

Representación: una ruta es una permutación r = [0, p1, p2, ..., pn] donde 0 es el almacén y {p1,...,pn} son las paradas de entrega en orden de visita.

**Función objetivo (a minimizar):**

f(r) = d(rn, r1) + ∑_{i=1}^{n-1} d(ri, r_{i+1}), con d(i,j) = distancia euclídea entre puntos.

Instancia utilizada: 1 almacén + 20 paradas = 21 puntos, coordenadas aleatorias en un plano 2D. Se utiliza la distancia euclídea entre cada par de puntos.

## 4. Vecindad

Para generar soluciones vecinas se emplean dos operadores complementarios:

- Intercambio (swap): se seleccionan dos paradas distintas y se intercambian sus posiciones.
- Inversión / 2-opt: se invierte el orden de un segmento aleatorio entre dos paradas, lo cual suele ser muy eficaz en TSP.

## 5. Algoritmos y parámetros

### 5.1 Hill Climbing

Hill Climbing con estrategia de mejor vecino (steepest ascent). En cada iteración explora todos los vecinos generados por swap e inversión, y si existe alguna mejora acepta el mejor de ellos; en caso contrario se detiene por convergencia (máximo sin mejora).

| Parámetro | Valor | Descripción |
|---|---|---|
| max_iter | 10 000 | Máximo de iteraciones |
| max_sin_mejora | 150 | Criterio de parada por estancamiento |
| Vecindad | swap + 2-opt | Operadores explorados |

### 5.2 Simulated Annealing

Simulated Annealing acepta mejoras siempre y acepta empeoramientos con probabilidad P(Δ) = exp(-Δ/T). La temperatura disminuye exponencialmente con factor alpha.

| Parámetro | Valor | Descripción |
|---|---|---|
| T0 (Temperatura inicial) | 12.0 | Temperatura inicial |
| Tf (Temperatura final) | 0.001 | Temperatura de parada |
| alpha | 0.995 | Factor de enfriamiento |
| iter_por_T | 80 | Iteraciones por nivel de temperatura |
| Vecindad | swap o 2-opt (aleatorio) | Operador seleccionado aleatoriamente |

## 6. Gráficas de convergencia

La gráfica de convergencia muestra la evolución del mejor costo encontrado durante la búsqueda, partiendo de la misma ruta inicial aleatoria.

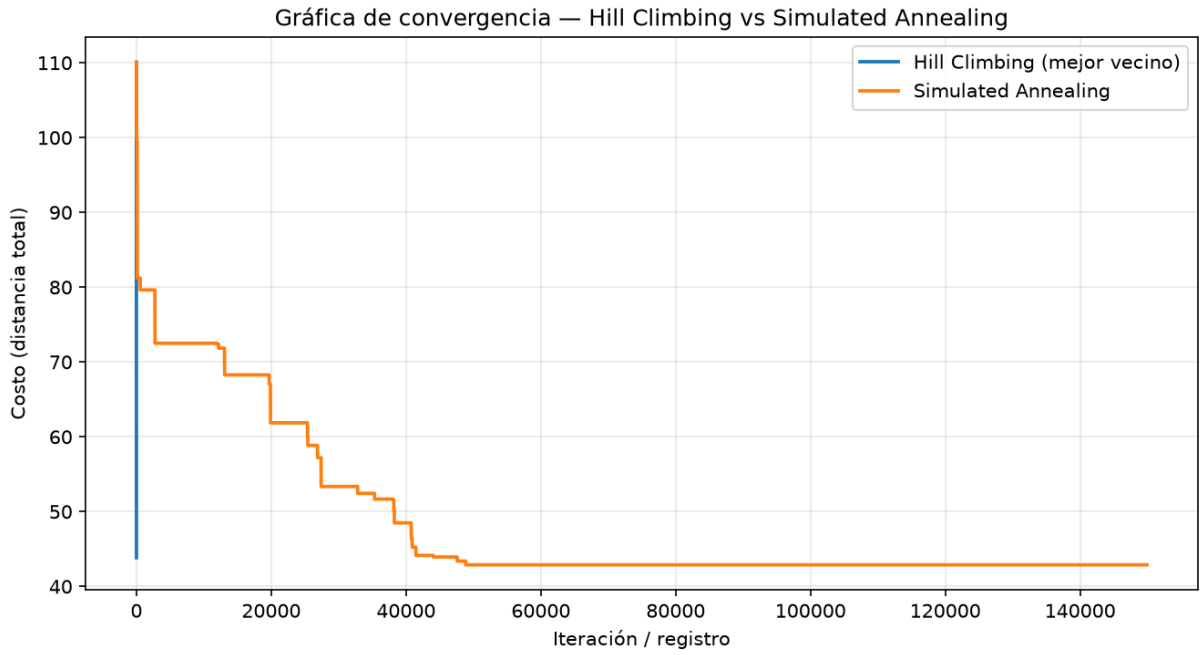



*Figura 1. Gráfica de convergencia — Hill Climbing vs Simulated Annealing*

## 7. Comparación de rutas y resultados

Se comparan las rutas obtenidas: inicial, Hill Climbing y Simulated Annealing.

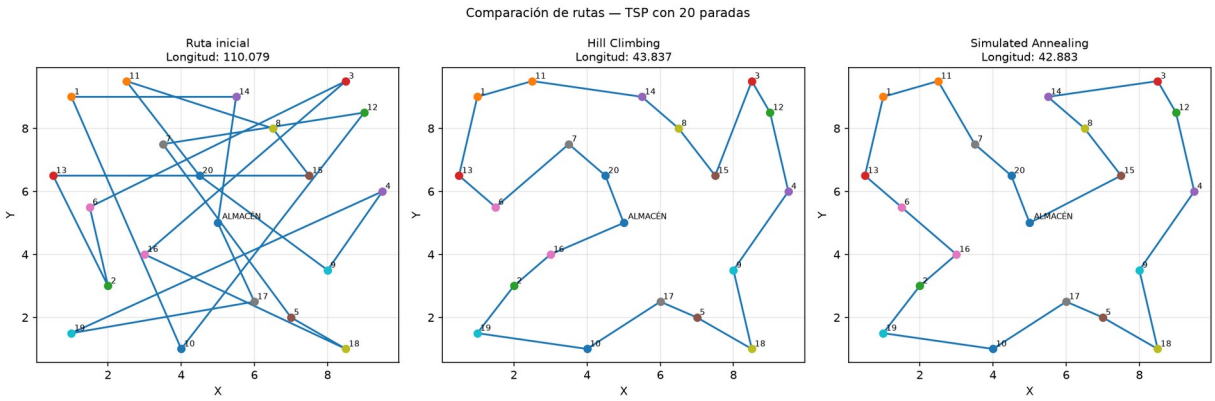



*Figura 2. Comparación visual de rutas (almacén + 20 paradas)*

También se realiza un análisis estadístico con 30 ejecuciones independientes partiendo de rutas aleatorias distintas para cada algoritmo.

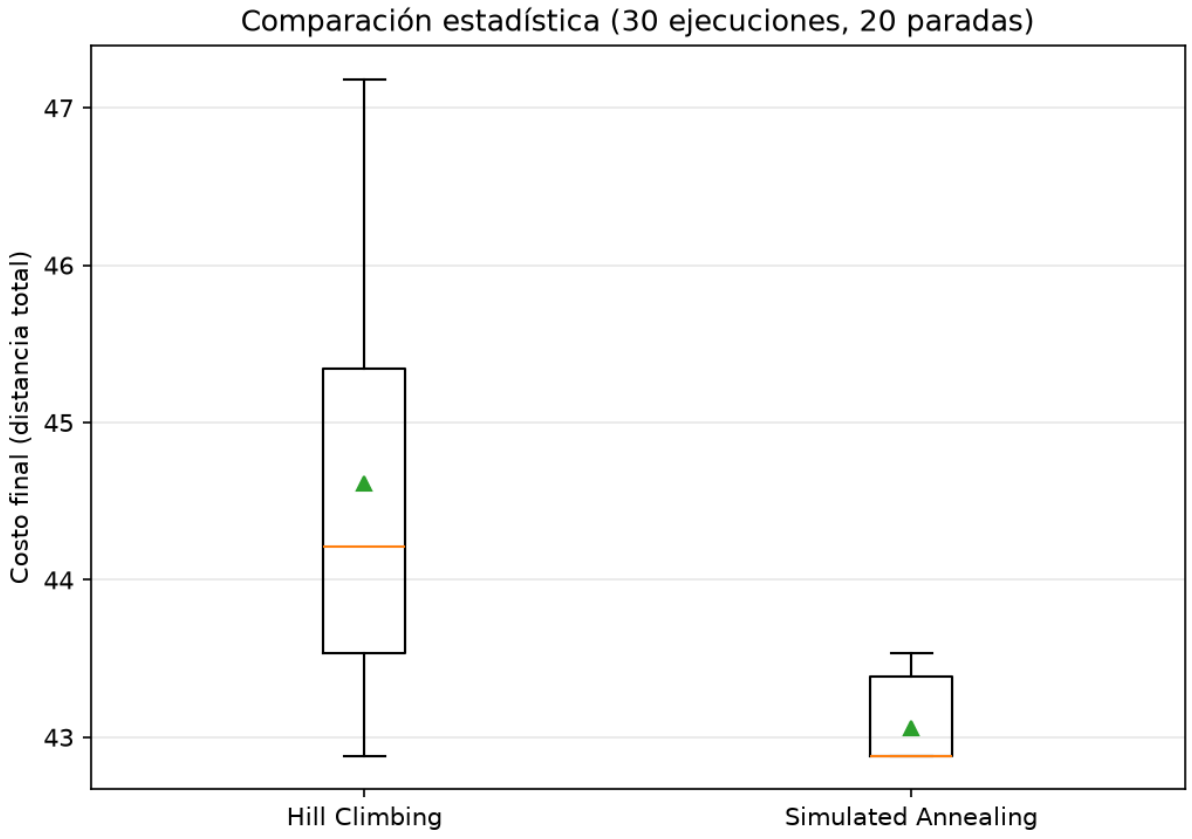



*Figura 3. Comparación estadística de 30 ejecuciones (boxplot)*

## 8. Resultados numéricos

| Configuración | Hill Climbing | Simulated Annealing |
|---|---|---|
| Misma ruta inicial aleatoria | 43.837 | 42.883 |

| | | |
|---|---|---|
| Con ruta inicial (vecino más próximo) | 42.883 | 42.883 |
| 30 ejecuciones — mínimo | 42.883 | 42.883 |
| 30 ejecuciones — media | 44.613 | 43.058 |
| 30 ejecuciones — mediana | 44.209 | 42.883 |
| 30 ejecuciones — desviación típica | 1.148 | 0.275 |
| 30 ejecuciones — máximo | 47.181 | 43.531 |

## 9. Análisis y conclusiones

- Hill Climbing es rápido y eficaz cuando parte de una buena solución inicial, pero tiende a quedar atrapado en óptimos locales si parte de una ruta aleatoria.
- Simulated Annealing explora más el espacio de búsqueda al aceptar soluciones peores con cierta probabilidad, lo que le permite escapar de óptimos locales.
- Con rutas iniciales aleatorias, Simulated Annealing obtiene una media mucho mejor (43.058) frente a Hill Climbing (44.613) y una desviación típica considerablemente menor (0.275 vs 1.148), lo que demuestra mayor robustez.
- Ambos algoritmos alcanzan el mismo óptimo encontrado (42.883) en las mejores ejecuciones, pero SA es más consistente entre ejecuciones.
- La combinación de operadores (intercambio + 2-opt) resulta adecuada para este TSP, permitiendo mejoras significativas respecto a la ruta inicial.

En suma, para este problema, Simulated Annealing resulta más adecuado cuando no se dispone de una buena solución inicial, mientras que ambos son equivalentes partiendo de heurísticas constructivas como el vecino más próximo.

## 10. Referencias

**[1]** RUSSELL, S. y NORVIG, P. (2021). Artificial Intelligence: A Modern Approach, 4.ª ed. Pearson. Capítulo 4 (Búsqueda local).

**[2]** KIRKPATRICK, S., GELATT, C. D. y VECCHI, M. P. (1983). Optimization by Simulated Annealing. Science, 220(4598), 671–680. DOI: 10.1126/science.220.4598.671.

**[3]** METROPOLIS, N. et al. (1953). Equation of State Calculations by Fast Computing Machines. The Journal of Chemical Physics, 21(6), 1087–1092. DOI: 10.1063/1.1699114.

**[4]** PYTHON SOFTWARE FOUNDATION (s. f.). Python Documentation. Disponible en: https://docs.python.org/3/

**[5]** HUNTER, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. DOI: 10.1109/MCSE.2007.55.

**[6]** OPENCODE (2026). OpenCode: agente de inteligencia artificial para programación y análisis [software]. Disponible en: https://opencode.ai. Consultado el 22 de octubre de 2026.

### Declaración de uso de inteligencia artificial

Para la elaboración de esta memoria se utilizó la herramienta de inteligencia artificial OpenCode [6], como apoyo en la implementación, depuración del código Python, generación de gráficas y maquetación de los archivos Word y PDF. La selección del problema, la calibración de parámetros, la verificación de resultados

y la revisión del contenido corresponden a la persona estudiante, quien asume la responsabilidad sobre el material entregado.

# Anexo A — Listado del código fuente

*Código implementado en la carpeta `codigo/`.*

## A.1 problema_tsp.py — Función objetivo y datos

```
def longitud_ruta(ruta: List[int]) -> float:
    """Longitud total del recorrido (almacén -> ... -> almacén)."""
    total = 0.0
    for a, b in zip(ruta, ruta[1:]):
        total += MATRIZ_DIST[a, b]
    if ruta:
        total += MATRIZ_DIST[ruta[-1], ruta[0]]
    return float(total)
```

## A.2 busqueda_local.py — Hill Climbing

```
def hill_climbing(
    ruta_inicial: List[int],
    rng: random.Random,
    max_iter: int = 10000,
    estrategia: str = "mejor_vecino",
) -> Tuple[List[int], float, List[float], int]:
    """
    Hill Climbing con estrategia 'mejor vecino' (steepest ascent).
    Explora intercambios y 2-opt, acepta únicamente mejoras.
    """
    ruta = list(ruta_inicial)
    actual = longitud_ruta(ruta)
    hist = [actual]
    evals = 0
    sin_mejora = 0
    max_sin_mejora = 150

    for _ in range(max_iter):
        mejor_ruta, mejor_c = None, actual
        # Explorar operadores de vecindad
        for i in range(1, N - 1):
            for j in range(i + 1, N):
                cand = ruta[:i] + ruta[i:j + 1][::-1] + ruta[j + 1:]
                c = longitud_ruta(cand)
                evals += 1
                if c < mejor_c - 1e-12:
                    mejor_c, mejor_ruta = c, cand

        for i in range(1, N - 1):
            for j in range(i + 1, N):
                cand = list(ruta)
                cand[i], cand[j] = cand[j], cand[i]
                c = longitud_ruta(cand)
                evals += 1
                if c < mejor_c - 1e-12:
                    mejor_c, mejor_ruta = c, cand

        if mejor_ruta is None or mejor_c >= actual - 1e-12:
            sin_mejora += 1
```

```
            if sin_mejora >= max_sin_mejora:
                break
            continue

        sin_mejora = 0
        ruta, actual = mejor_ruta, mejor_c
        hist.append(actual)

    return ruta, actual, hist, evals
```

## A.3 busqueda_local.py — Simulated Annealing

```
def simulated_annealing(
    ruta_inicial: List[int],
    rng: random.Random,
    T0: float = 12.0,
    Tf: float = 0.001,
    alpha: float = 0.995,
    iter_por_T: int = 80,
) -> Tuple[List[int], float, List[float], int]:
    """
    Simulated Annealing.
    Acepta mejoras siempre y empeoramientos con probabilidad exp(-ΔE/T).
    """
    ruta = list(ruta_inicial)
    actual = longitud_ruta(ruta)
    mejor, mejor_c = list(ruta), actual
    hist = [mejor_c]
    evals = 0
    T = T0

    while T > Tf:
        for _ in range(iter_por_T):
            if rng.random() < 0.5:
                cand = generar_vecino_intercambio(ruta, rng)
            else:
                cand = generar_vecino_inversion(ruta, rng)
            c = longitud_ruta(cand)
            evals += 1
            delta = c - actual
            if delta < 0.0 or rng.random() < math.exp(-delta / T):
                ruta, actual = cand, c
                if c < mejor_c - 1e-12:
                    mejor, mejor_c = list(cand), c
            hist.append(mejor_c)
        T *= alpha

    return mejor, mejor_c, hist, evals
```